

Canonical Huffman Coding Demonstration Input

Section 1: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information.

The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 2: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated

patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 3: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 4: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented

organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 5: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 6: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the

canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 7: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 8: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree,

converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 9: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 10: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 11: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an

educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 12: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 13: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the

project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 14: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 15: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code

lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 16: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 17: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of

every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 18: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 19: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 20: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been

compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 21: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 22: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented

organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 23: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 24: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code

lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 25: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 26: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of

every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 27: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 28: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 29: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been

compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 30: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 31: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented

organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 32: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 33: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression,

the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 34: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 35: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information.

The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 36: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 37: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 38: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated

patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 39: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 40: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented

organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 41: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 42: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression,

the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 43: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 44: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of

every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 45: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 46: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 47: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated

patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 48: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 49: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level

operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 50: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 51: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores

the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 52: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 53: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original

information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 54: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in

modern compression systems.

Section 55: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 56: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated

patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 57: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 58: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level

operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 59: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 60: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression,

the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 61: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 62: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original

information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 63: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in

modern compression systems.

Section 64: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 65: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on

the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 66: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 67: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming,

binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 68: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 69: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a

compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 70: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 71: Priority Queues. This project demonstrates how a file can be

represented more efficiently without changing its original information.

The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 72: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in

modern compression systems.

Section 73: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 74: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on

the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 75: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 76: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level

operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 77: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 78: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a

compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 79: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 80: Testing and Validation. This project demonstrates how a file

can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 81: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration

of practical lossless compression techniques used as building blocks in modern compression systems.

Section 82: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 83: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and

extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 84: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 85: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design

emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 86: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 87: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated

codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 88: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 89: Greedy Algorithms. This project demonstrates how a file can

be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 90: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration

of practical lossless compression techniques used as building blocks in modern compression systems.

Section 91: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 92: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on

the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 93: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 94: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design

emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 95: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 96: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree,

converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 97: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 98: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 99: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an

educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 100: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 101: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the

project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 102: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 103: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code

lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 104: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 105: Greedy Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of

every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 106: C++ Standard Template Library. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 107: File Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 108: Computer Networks. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been

compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 109: Operating Systems. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 110: Software Engineering. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented

organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 111: Performance Analysis. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 112: Testing and Validation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression,

the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 113: Data Compression. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 114: Lossless Compression. This project demonstrates how a file can be represented more efficiently without changing its original

information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 115: Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information.

The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 116: Canonical Huffman Coding. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 117: Binary File Processing. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated

patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 118: Bit Manipulation. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 119: Priority Queues. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level

operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.

Section 120: Trees in Algorithms. This project demonstrates how a file can be represented more efficiently without changing its original information. The implementation begins by counting the frequency of every byte in the input stream, constructs an optimal Huffman tree, converts the generated codes into canonical Huffman codes, writes a compact header, and stores the encoded bit stream. During decompression, the header is read, the canonical codes are reconstructed from code lengths, and the original file is recovered exactly. The design emphasizes modular programming, binary file handling, bit-level operations, efficient use of memory, and clean object-oriented organization. Each module has a single responsibility, making the project easier to understand, test, and extend. Performance depends on the distribution of symbols in the input; files containing repeated patterns generally compress much better than data that has already been compressed. This implementation serves as an educational demonstration of practical lossless compression techniques used as building blocks in modern compression systems.